Exam Scheduling: A Comparative Analysis of Algorithms

By Joseph Dye, Simon Glashauser, Nick Russell, and Mason Thomas

Problem Introduction

What is the Exam Scheduling Problem?

- Optimization problem that aims to create a conflict free exam timetable
- Arises from the need to organize exams efficiently while avoiding conflicts
  - No student scheduled for multiple exams at the same time
  - No exam in the same place at the same time
- Combinations based on number of exams, number of students, number of timeslots, and number of locations

Algorithms

- Brute Force (exhaustive)
- Greedy Algorithm
- Graph Coloring Algorithm
- Genetic Algorithm

Data Generation

- Required Data
  - Locations
  - Exams
  - Students
  - Timeslots (hard-coded)
- Python script used to generate data
  - Input count of data type
  - Outputs formatted JSON file
- Exam data is abstract, so this works
- Student data not readily public

# of Exams	# of Students	# of Locations	# of Time Slots
n	5n	n/20	20 (constant)

Solution Guidelines

- Hard Constraints
  - No exams at the same time in the same place
  - No students in multiple exams at the same time
- Algorithms rated based on runtime and number of conflicts
- Run on 2023 Apple MacBook Pro
  - Apple M2 Processor
    - 12-core CPU
    - 19-core GPU
  - 16GB RAM
- Compiled using gcc (algorithms written in C, wrapper written in Python)

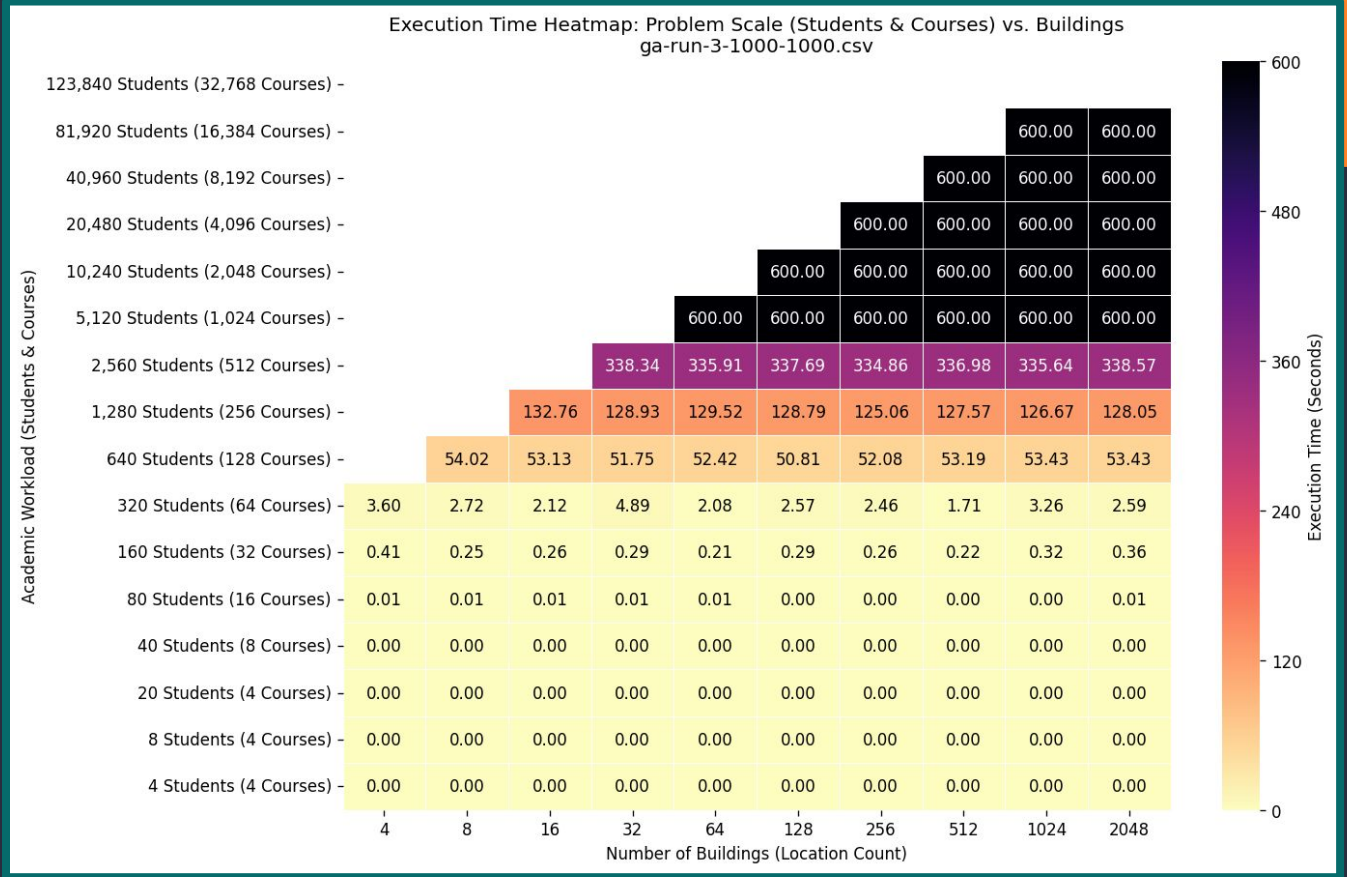
Live Demo

We will now go over a short demo showing our genetic algorithm.

Results: Introduction

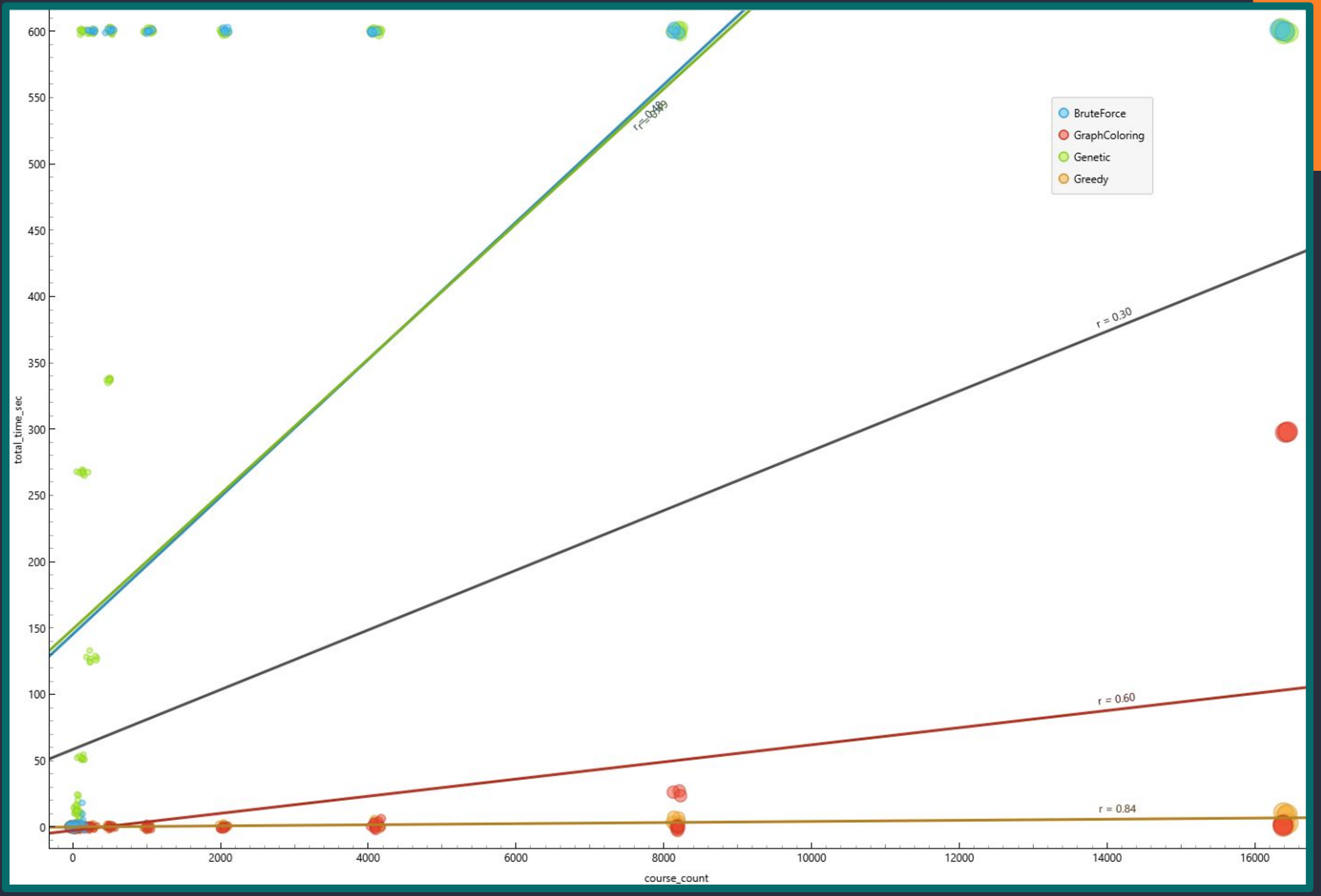
- Metrics
  - Runtime
  - Number of conflicts
- Basic Operations
  - Brute Force: Generate schedule permutation
  - Greedy: Course placed in graph
  - Graph Coloring: Coloring an edge
  - Genetic: Schedule generation per individual
- Iterations
  - 115 combinations (students, courses, locations)

Results: Analysis



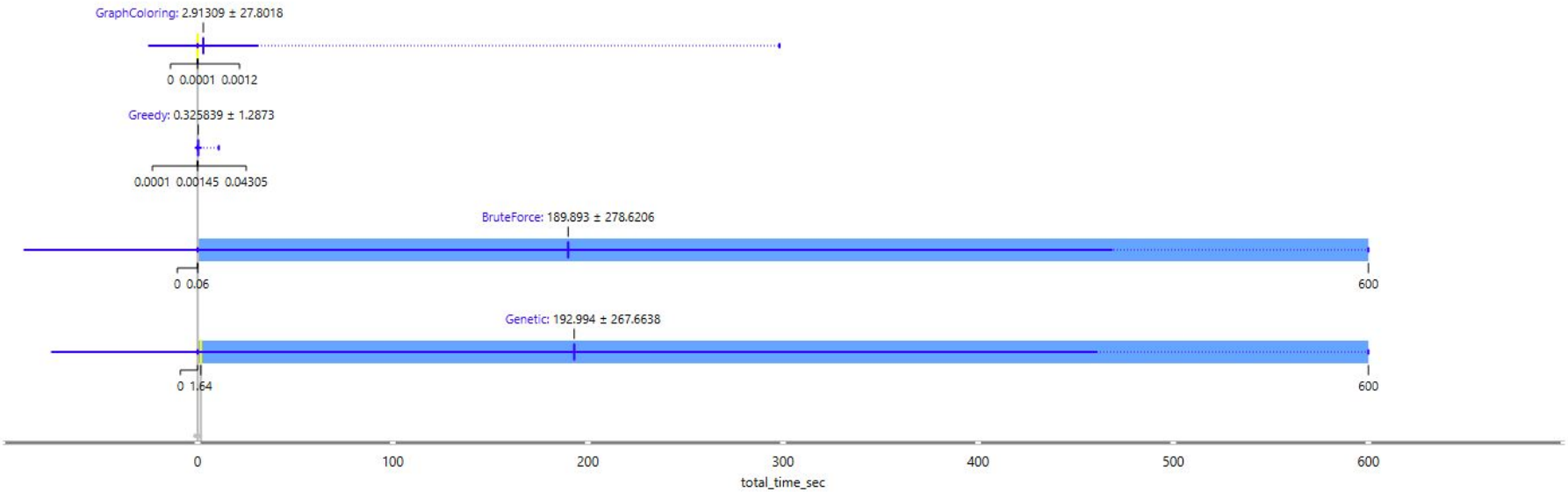
- Heat map for one of our genetic runs
- Population: 1000
- Generations: 1000
- Timeout of 600s

Results: Analysis



- Colors indicate categories
- Dot sizes based on location count
- 600 seconds is the timeout
- Brute Force and Genetic slower
- Greedy and Graph Coloring *much* faster

Results: Analysis



Comparative visualization of runtime

- Greedy and Graph coloring both near-instant
- Brute Force and Genetic have a much wider spread

Research Questions

1. How does adjusting the population size and generation count affect the quality of the output of the genetic algorithm, and is it worth computational costs?
2. In what cases would we expect the greedy algorithm to outperform the graph coloring algorithm?
3. How does decreasing the number of colors (timeslots) in a graph coloring problem impact the runtime and memory requirements of the algorithm?

Results: Research Questions

- Research Question Results:

1. Genetic Question
   a. Varied based on affected parameter
   b. Not advantageous to to increase pop/gens
2. Greedy vs. Graph-Coloring
   a. Graph Coloring surprisingly better at small n
   b. G.C. greatly explodes later
3. Graph-Coloring Time Slot Decrease
   a. Surprisingly, runtime increased
   b. Produced more conflicts

Real-World Applications

- If you could believe it, the Exam Scheduling Problem is used to schedule exams...

Other Uses

- Sports Tournament Scheduling
  - No teams having games at the same time/place
- Employee Work Schedules
  - No person scheduled to work in two different places at the same time
  - Multiple people not filling the same position
- Event Planning
  - No events overlap in both time and location
  - Multi-day events not scheduled at the same time

Future Improvements and Limitations

- Future Improvements:
  - Better to use real-world data where applicable rather than solely generated data
  - Further test resource limitations
  - Better tune parameters for the genetic algorithm
  - Create hybrid of Greedy and Graph Coloring
  - Multi-threading for Brute Force and Genetic
- Limitations:
  - Brute Force and Genetic consistently time out at large input sizes (600s)
  - Often schedules not found due to limited room availability

References

Aspnes, James. "BruteForce." *PineWiki*, 17 June 2014, www.cs.yale.edu/homes/aspnes/pinewiki/BruteForce.html.

"Brute Force Approach and Its Pros and Cons." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/dsa/brute-force-approach-and-its-pros-and-cons/.

Elen, Abdullah. "A COMPLETE SOLUTION to EXAM SCHEDULING PROBLEM: A CASE STUDY." *Kutahya Dumlupmar University Institute of Graduate Studies*, Journal of Scientific Reports - A, 14 Oct. 2021, dergipark.org.tr/en/download/article-file/2025907.

"Genetic Algorithms." *GeeksforGeeks*, GeeksforGeeks, 10 Feb. 2026, www.geeksforgeeks.org/dsa/genetic-algorithms/.

"Graph Coloring Using Greedy Algorithm." *GeeksforGeeks*, GeeksforGeeks, 23 Mar. 2025, www.geeksforgeeks.org/dsa/graph-coloring-set-2-greedy-algorithm/.

"Greedy Algorithms Tutorial." *GeeksforGeeks*, GeeksforGeeks, 22 Jan. 2026, www.geeksforgeeks.org/dsa/greedy-algorithms/.

"Introduction to Graph Coloring." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/dsa/graph-coloring-applications/.

Lemos, Alexandre Rosseto Rosseto. "Genetic Algorithm and Its Practicality in Machine Learning." *Towards Data Science*, 5 Mar. 2025, towardsdatascience.com/genetic-algorithm-6aefd897f1ac/.

W3Schools. "DSA Greedy Algorithms." *W3Schools Online Web Tutorials*, www.w3schools.com/dsa/dsa_ref_greedy.php.

Artificial Intelligence Use

Claude: Used for generating C-based algorithms and the conversion of python algorithms into C

ChatGPT: Used for inquiry and editing of the used algorithms

GitHub Copilot: Used for debugging and editing file paths in the algorithms